

Pre-Training Fundamentals

How large language models learn from the world — objectives, data, and implementation

6 Concepts

67 minutes

5 Theory

1 Coding Lab

B2–B3 Bloom's

L2–L3 Difficulty

Learning Objectives

What you will be able to do after this sub-module

B2 Understand

Distinguish pre-training from fine-tuning, instruction tuning, and RLHF — and explain why each phase exists

B3 Apply

Explain the CLM autoregressive factorisation and trace loss computation token-by-token

B3 Apply

Contrast causal vs. masked LM attention — and identify which architecture suits which use case

B3 Apply

Describe the full data pipeline from raw web crawl to tokenised training corpus

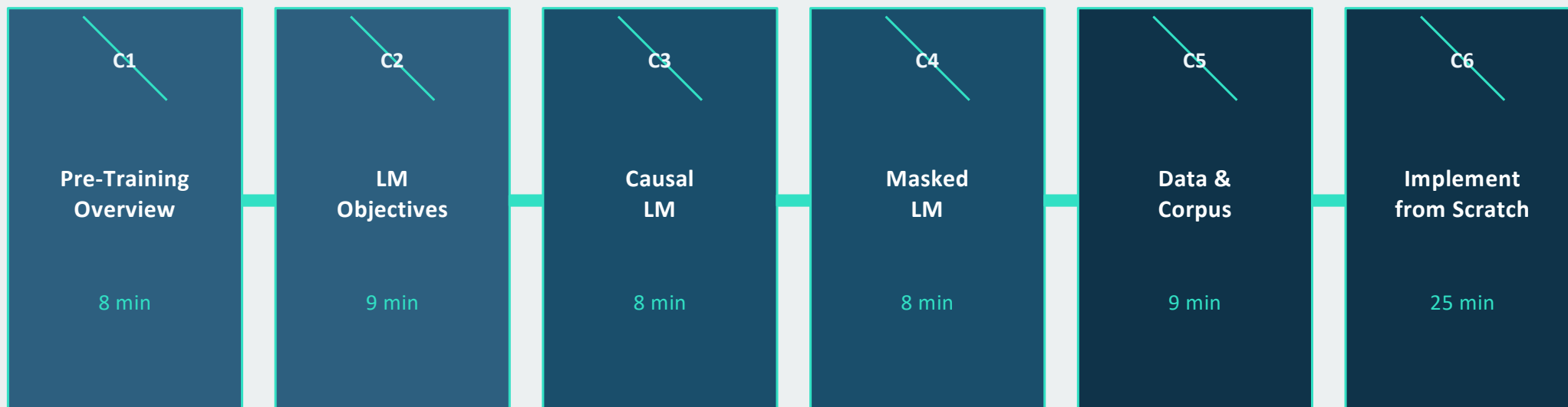
B3 Implement

Build and train a ~120M parameter causal LM from scratch in PyTorch with gradient accumulation and LR warmup

Sub-Module 9.1 Roadmap



Six concepts, one thread: from objective to implementation



Theory (C1–C5)

C O N C E P T

Pre-Training Overview

Phase 1 of LLM development — learning the world from raw text

1

What is Pre-Training?

The foundation of every large language model

DEFINITION

Pre-training is the first phase of LLM development. A model learns general-purpose language representations by predicting text tokens from a massive, diverse corpus using self-supervised objectives — no human labels required.

- ✓ Self-supervised — labels come from the data itself
- ✓ Massive scale — billions to trillions of tokens
- ✓ General capability — not tuned for any task
- ✓ Transferable — checkpoint reused for downstream tasks

LLM Development Phases

PHASE 1

Pre-Training

Self-supervised · Massive corpus · General capability

PHASE 2

Fine-Tuning / SFT

Supervised · Curated examples · Task alignment

PHASE 3

RLHF / DPO

Reward signal · Human feedback · Preference alignment

Comparing the Three Training Phases

How pre-training differs from fine-tuning and RLHF

Dimension	Pre-Training	Fine-Tuning / SFT	RLHF / DPO
Supervision	Self-supervised (no labels)	Supervised (curated examples)	Reward signal (human preferences)
Data scale	300B – 15T tokens	1K – 1M examples	10K – 100K comparisons
Objective	Predict next/masked token	Minimise task loss	Maximise reward model score
Compute	Months on thousands of GPUs	Hours–days on 1–8 GPUs	Days on 8–64 GPUs
Output	General-purpose checkpoint	Task-specialized model	Human-preference-aligned model

Pre-Training at Scale — Frontier Benchmarks

The compute investment that produces general intelligence

GPT-3

Tokens

300B

Params

175B

GPUs

~1,000 A100s

Time

~1 month

GPT-4

Tokens

~1T

Params

~1.8T (MoE)

GPUs

Thousands of A100s

Time

Several months

Llama 3.1 405B

Tokens

15.6T

Params

405B

GPUs

16,000 H100s

Time

Months

Gemma 2 27B

Tokens

13T

Params

27B

GPUs

Hundreds of TPUv5

Time

Weeks

Why Pre-Trained Representations Transfer

Shared linguistic structure underlies all downstream tasks



The Open-Weight Model Ecosystem



Pre-trained checkpoints you can download and use today

Model Family	Org	Sizes	Tokens	Architecture	Licence
Llama 3.1	Meta	8B, 70B, 405B	15.6T	Decoder-only (GQA)	Llama 3 Community
Mistral 7B	Mistral AI	7B	Unknown	Decoder (Sliding Window)	Apache 2.0
Falcon 180B	TII UAE	7B, 40B, 180B	3.5T	Decoder (Multi-Query)	Falcon Licence
Gemma 2	Google	2B, 9B, 27B	13T	Decoder (logit softcap)	Gemma ToS
Phi-3	Microsoft	3.8B, 7B, 14B	3.3T	Decoder-only	MIT

C O N C E P T

Language Modeling Objectives

The self-supervised signals that drive pre-training

2

Two Dominant Pre-Training Objectives

The choice of objective determines architecture and capability

Causal Language Modeling (CLM)

Predict the NEXT token given all previous tokens

- ✓ $P(x_t \mid x_1, \dots, x_{t-1})$ for every position t
- ✓ Causal (unidirectional) attention mask
- ✓ Decoder-only architecture
- ✓ Naturally supports autoregressive generation
- ✓ Models: GPT-2, GPT-3, Llama 3, Mistral, Falcon

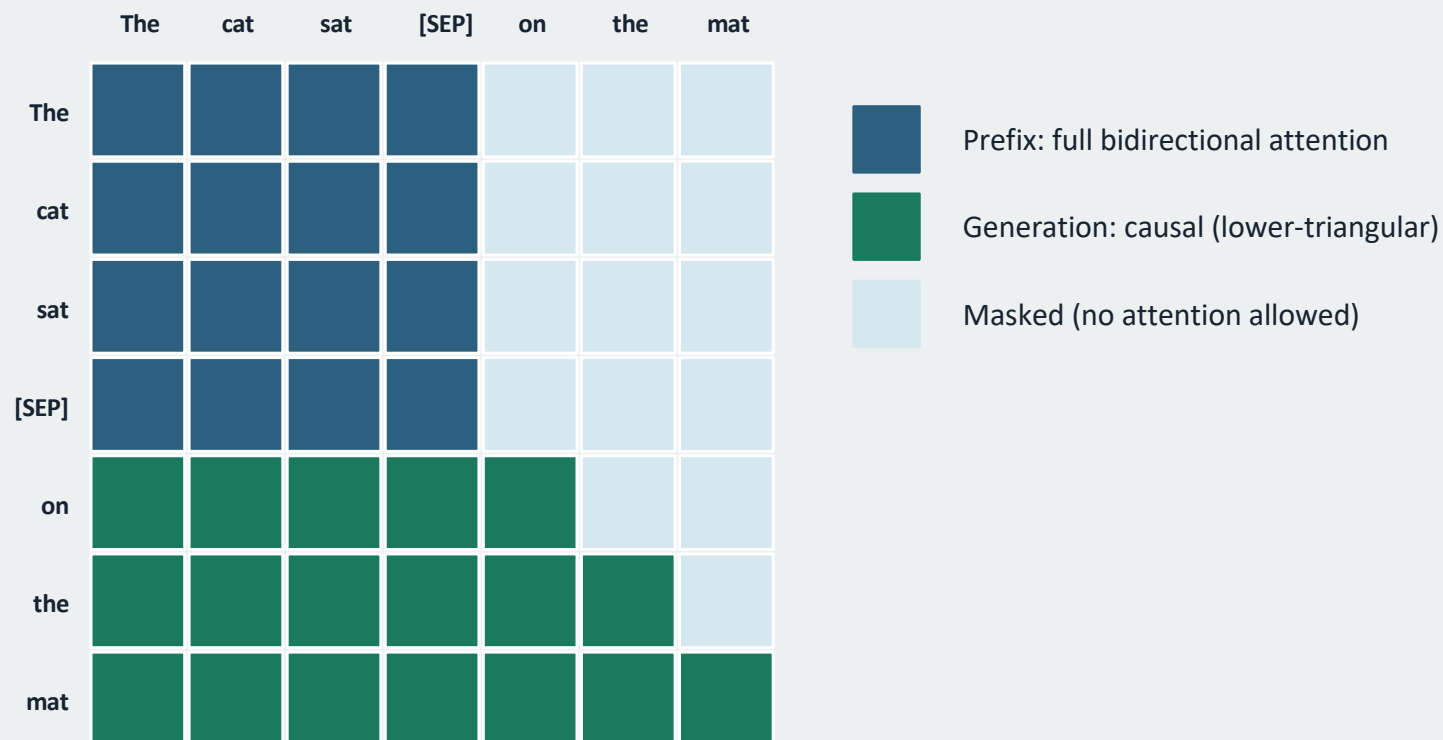
Masked Language Modeling (MLM)

Predict randomly MASKED tokens from full bidirectional context

- ✓ 15% of tokens masked; model predicts [MASK]
- ✓ Bidirectional attention — sees full sentence
- ✓ Encoder-only architecture
- ✓ Rich contextual embeddings for classification
- ✓ Models: BERT, RoBERTa, DeBERTa, ELECTRA

The Prefix LM — A Hybrid Objective

Bidirectional prefix + causal generation: T5, PaLM, Flan-T5



Prefix LM enables encoder-style understanding of the prompt and decoder-style generation — used in T5, PaLM, and Flan-T5.

Objective → Architecture → Use Case

How training objective shapes everything downstream

Attribute	Causal LM (CLM)	Masked LM (MLM)	Prefix LM
Attention direction	Left-to-right only	Bidirectional	Bidirectional prefix + causal suffix
Architecture	Decoder-only	Encoder-only	Encoder-decoder
Training signal	Predict next token	Predict masked tokens	Both objectives
Can generate text?	✓ Natively	✗ Not directly	✓ Via decoder
Best downstream use	Generation, reasoning, few-shot prompting	Classification, NER, embeddings, RAG	Seq2seq, summarisation, translation
Example models	GPT-4, Llama 3, Mistral, Falcon	BERT, RoBERTa, DeBERTa, ELECTRA	T5, Flan-T5, PaLM

C O N C E P T

Causal Language Modeling

Autoregressive factorisation — the engine behind GPT-4, Llama, and Mistral

3

Autoregressive Factorisation

The probability chain-rule behind every CLM model

$$P(x_1, x_2, \dots, x_n) = \prod P(x_t \mid x_1, \dots, x_{t-1})$$

Worked example: "The cat sat"

Step 1

$P(\text{The}) = 0.021$ (prior probability in vocabulary)

Step 2

$P(\text{cat} \mid \text{The}) = 0.018$ (given 'The', cat is plausible)

Step 3

$P(\text{sat} \mid \text{The, cat}) = 0.042$ (given 'The cat', sat is more likely)

Joint P

$P(\text{The, cat, sat}) = 0.021 \times 0.018 \times 0.042 \approx 0.0000159$

The Causal Attention Mask

Why training is parallel but generation is sequential

	The	cat	sat	on	mat
The	1	$-\infty$	$-\infty$	$-\infty$	$-\infty$
cat	1	1	$-\infty$	$-\infty$	$-\infty$
sat	1	1	1	$-\infty$	$-\infty$
on	1	1	1	1	$-\infty$
mat	1	1	1	1	1

Key properties

Lower-triangular

Each position i attends only to positions $\leq i$

$-\infty$ masking

$\text{Softmax}(-\infty) = 0$ — future tokens contribute zero

Parallel training

All positions computed in one forward pass

Sequential inference

Each new token generated one at a time

Cross-Entropy Loss Over the Vocabulary

How the model measures and minimises its prediction error

$$\mathcal{L} = - \sum_t \log P(x_t \mid x_1, \dots, x_{t-1}; \theta)$$

If $P(\text{correct}) = 1.0$

Loss = 0
Perfect prediction

If $P(\text{correct}) = 0.5$

Loss = 0.69
Good prediction

If $P(\text{correct}) = 0.1$

Loss = 2.30
Poor prediction

If $P(\text{correct}) = 0.01$

Loss = 4.61
Random guess

Perplexity (PPL) = $\exp(\mathcal{L} / T)$ — The standard metric for comparing LM training progress. Lower PPL = better model.

Note: tokeniser quality directly affects CLM stability — a poor tokeniser assigns low probability to valid token sequences, artificially inflating perplexity.

Teacher Forcing vs. Free-Running Generation

A critical train-test distribution mismatch

TRAINING — Teacher Forcing

Input: The [true] cat [true] sat

Target: cat sat on

Loss: computed at every position

Key: Always feeds GROUND TRUTH as input

INFERENCE — Free-Running

Input: The → model predicts 'cat'

Next: 'The cat' → model predicts 'sat'

Error: If model predicted 'dog', next context is corrupted

Key: Feeds MODEL'S OWN output

Exposure Bias — The Train-Test Distribution Gap

Why CLM models can degrade with long generation chains

Exposure Bias: During training the model always sees ground-truth tokens. At inference it sees its own (potentially wrong) output. Errors compound — each mistake shifts the context further from the training distribution.

Mitigation Strategies

Scheduled Sampling

Gradually replace teacher-forced inputs with model predictions during training — curriculum from teacher to self-feeding.

REINFORCE / RL Fine-Tuning

Treat generation as a sequential decision problem; reward entire sequences rather than individual tokens.

Minimum Risk Training

Optimise expected task metric (BLEU, ROUGE) computed on sampled sequences directly.

DPO / RLHF

Align generation quality using human preference comparisons — indirectly reduces exposure bias by penalising low-quality completions.

C O N C E P T

Masked Language Modeling

BERT's bidirectional objective — the foundation of encoder models and RAG

4

BERT's 80/10/10 Masking Rule

Why masking strategy matters for pre-training quality

80%

[MASK] token

Example: The cat [MASK] on the mat

Model must predict from context — primary training signal

10%

Random token

Example: The cat danced on the mat

Prevents model from ignoring non-[MASK] positions entirely

10%

Unchanged

Example: The cat sat on the mat

Ensures token representations remain useful for classification

Only 15% of tokens are selected for prediction — the rest are kept unmodified so the model also learns context for non-masked tokens.

Bidirectional vs. Causal Attention Masks

The structural difference that separates BERT from GPT

Causal (CLM)

	The	cat	sat	on
The	✓	✗	✗	✗
cat	✓	✓	✗	✗
sat	✓	✓	✓	✗
on	✓	✓	✓	✓

Can only see past tokens — enables generation

Bidirectional (MLM)

	The	cat	sat	on
The	✓	✓	✓	✓
cat	✓	✓	✓	✓
sat	✓	✓	✓	✓
on	✓	✓	✓	✓

Sees all tokens — richer contextual embeddings

Next Sentence Prediction — Introduced and Deprecated

A lesson in unnecessary pre-training objectives

2018

BERT
(Devlin et al.)

MLM + NSP:
predict if Sentence B follows
Sentence A

2019

RoBERTa
(Liu et al.)

Dropped NSP entirely.
MLM-only + longer training →
better on all benchmarks

2019

ALBERT
(Lan et al.)

Replaced NSP with Sentence
Order Prediction (SOP)

2020+

DeBERTa, ELECTRA, etc.

All omit NSP.
MLM remains the dominant
encoder objective

Dynamic Masking — Why It Improves MLM Training

RoBERTa's key innovation over original BERT

BERT — Static Masking

- ✓ Corpus tokenised ONCE before training
- ✓ Mask pattern fixed per sequence
- ✗ Same 15% tokens masked every epoch
- ✗ Model can memorise which positions are masked
- ✗ Leads to suboptimal generalisation

RoBERTa — Dynamic Masking

- ✓ Masking applied on-the-fly each epoch
- ✓ Different 15% selected each training pass
- ✓ Same token sees different contexts across epochs
- ✓ Model cannot rely on positional shortcut
- ✓ Better downstream performance across all tasks

RoBERTa result: dynamic masking alone recovered ~1 PPL point in perplexity compared to BERT's static masking, with identical architecture and data.

Why MLM Models Cannot Generate Text Natively

The architectural constraint that separates BERT from GPT

MLM models have no autoregressive structure — they predict masked tokens given BOTH left and right context. At generation time, there is no "future" to attend to, making token-by-token generation undefined.

Workarounds and Related Approaches

Iterative Masked Generation

BERT-style models can generate text by starting with all [MASK] tokens and iteratively refining — used in BERT-gen and CMLM. Slow but possible.

Encoder-Decoder (T5)

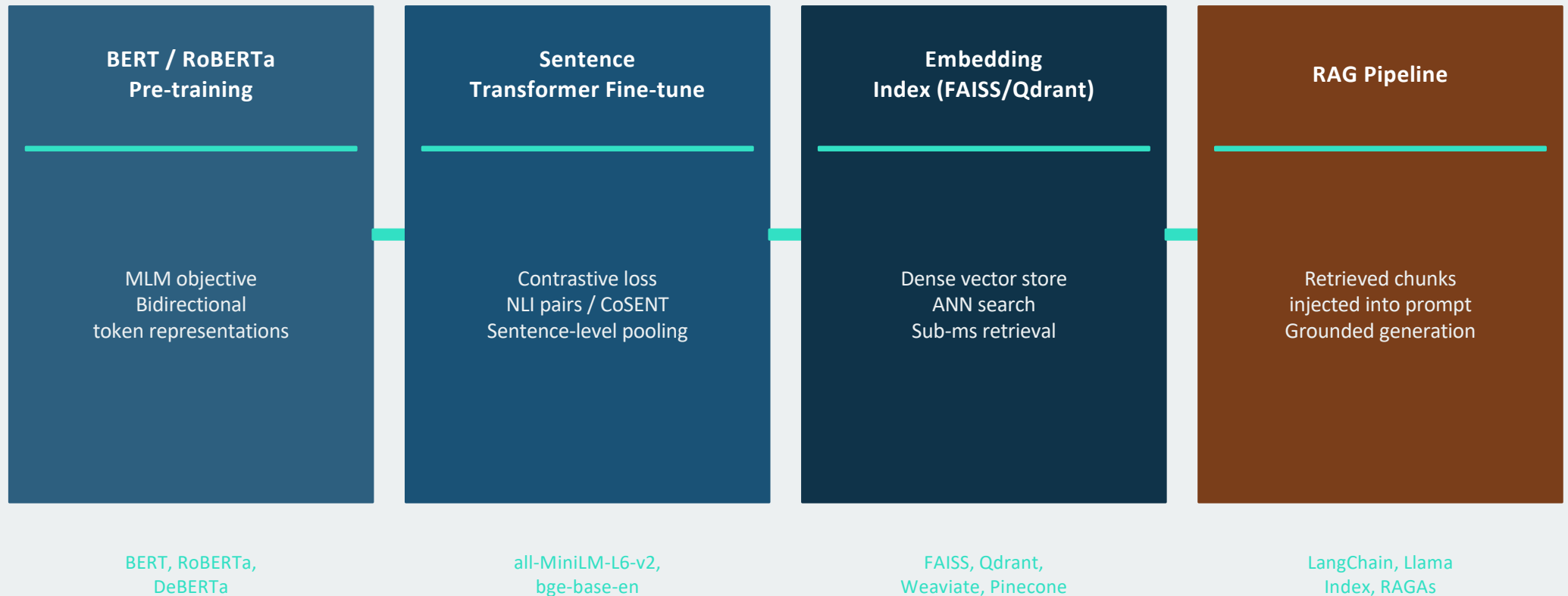
Add a causal decoder on top of the bidirectional encoder — the encoder reads the prompt with full attention, the decoder generates autoregressively.

Sentence Transformers

MLM representations are pooled into sentence embeddings for similarity and retrieval — this is the dominant MLM use case in production RAG pipelines.

MLM Embeddings → Sentence Transformers → RAG

The production path from BERT pre-training to retrieval-augmented generation



C O N C E P T

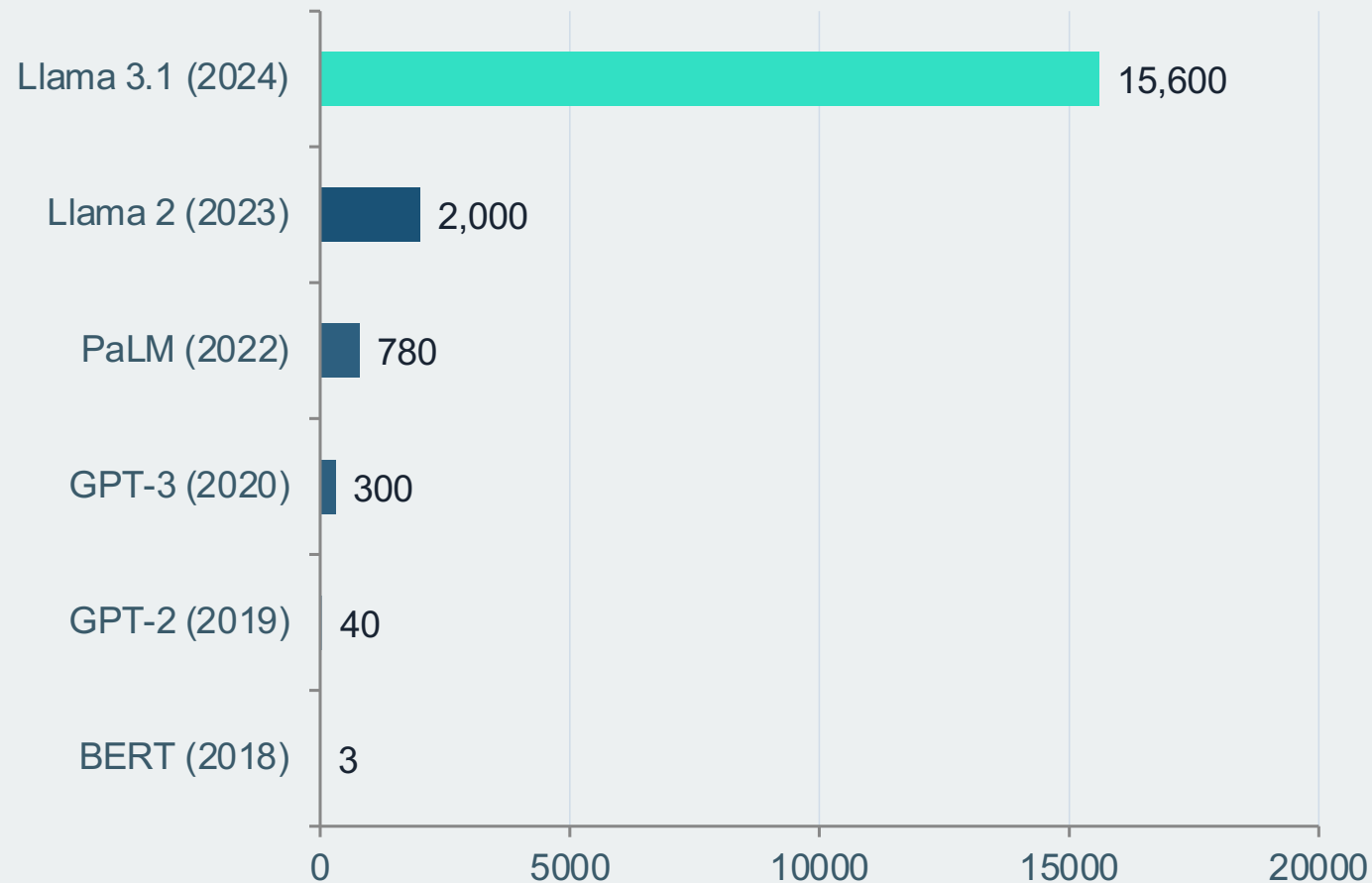
Data Collection & Corpus Construction

From the raw web to a tokenised pre-training corpus

5

Pre-Training Data Scale — Token Counts

The scale gap between frontier and open-weight models



3.3B

BERT
(2018)

300B

GPT-3
(2020)

15.6T

Llama 3.1
(2024)

Pre-Training Data Sources

Where the tokens come from — and what capability each confers

Source	Examples	Scale	Capability Contribution
Web text (Common Crawl)	C4, OSCAR, RefinedWeb	Trillions of tokens	General language, world knowledge, diverse styles
Books	Project Gutenberg, Books3, Pile of Law	~100B tokens	Long-range coherence, narrative, legal/formal text
Code	GitHub, The Stack, StarCoder data	~500B tokens	Reasoning, structured generation, tool use
Scientific papers	arXiv, PubMed, Semantic Scholar	~50B tokens	Formal reasoning, domain knowledge, citations
Wikipedia	English + multilingual	~20B tokens	Factual knowledge, encyclopaedic coverage
Curated datasets	The Pile, RedPajama, Dolma, ROOTS	Varies	Mixed — controlled quality, reproducible recipes

Deduplication — Exact & Near-Duplicate Removal

Why duplicate data hurts training and how to remove it

Why It Matters

- ✓ Common Crawl contains millions of near-identical boilerplate pages
- ✓ Models memorise duplicated content verbatim → privacy risk
- ✓ Duplicates inflate effective dataset size → misleading PPL
- ✓ LLM.int8() study: dedup alone improved Chinchilla by 1.5 PPL

Exact Deduplication

1. Hash each document (MD5, SHA-256)
 2. Build hash set — $O(1)$ lookup
 3. Discard any document whose hash already exists
- Coverage: catches byte-identical copies only

Near-Duplicate (MinHash LSH)

1. Shingling — split doc into k-grams ($k = 5-10$ words)
 2. MinHash — hash each k-gram, keep minimum per band
 3. LSH — bucket similar MinHash signatures together
 4. Jaccard similarity threshold (typically 0.8) → discard
- Coverage: finds paraphrased / re-ordered near-copies

Quality Filtering Pipeline

From raw web crawl to high-quality training tokens



Toxicity filtering applied as an additional pass before or after classifier filtering — typically a fine-tuned classifier trained on labelled toxic content datasets (Jigsaw, HatEval).

Toxicity Filtering in Pre-Training Data

Preventing the model from learning harmful patterns at source

Definition: Toxicity filtering removes or downweights documents containing hate speech, explicit violence, sexual content, and harassment from the pre-training corpus before a model ever sees them.

Common Approaches

Keyword blocklists

Fast, $O(1)$ lookup per token. High precision but poor recall — slang and coded language evade blocklists entirely.

fastText classifier (Jigsaw dataset)

Binary text classifier trained on Jigsaw toxic comment dataset. Scores each document 0–1. Threshold ≈ 0.7 . $\sim 94\%$ precision at scale.

Perspective API (Google)

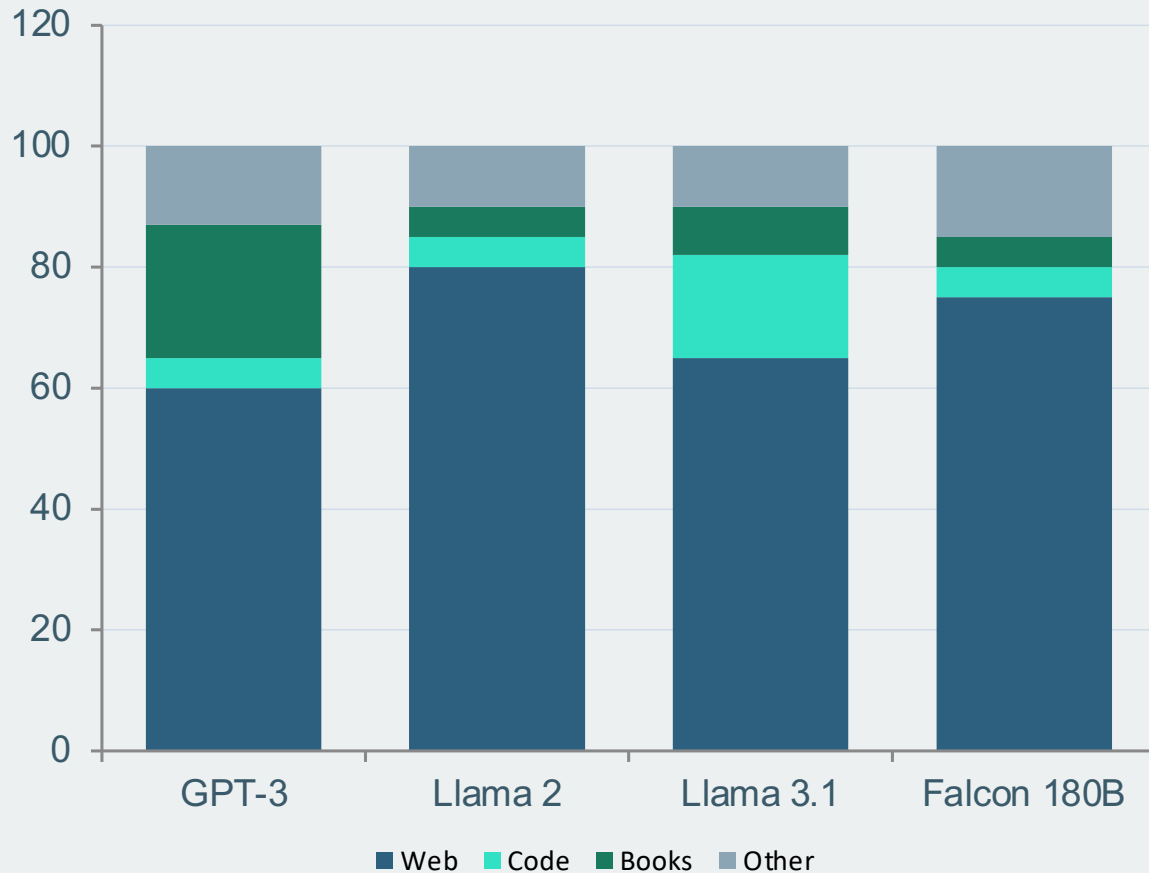
Neural toxicity scorer. High accuracy, handles nuanced hate speech, but rate-limited — impractical for trillion-token corpora.

Human auditing (sample)

Random sample of 1,000–10,000 documents reviewed post-filter. Confirms filter quality and calibrates threshold.

Data Mixing Ratios & Capability Profiles

The proportion of each source shapes what the model can do



Source ↑	Capability boost
More Code	Stronger reasoning, tool use, structured output
More Books	Better long-context coherence, narrative generation
More Web	Broader knowledge, better few-shot generalisation
More Science	Stronger math, formal reasoning, citation accuracy

Data Governance — Licensing, PII & Copyright



The legal and ethical boundaries of pre-training data

Licensing

Not all public data is free to train on.

Creative Commons (CC-BY, CC0) = generally safe.

CC-BY-NC = non-commercial use only.

All Rights Reserved = requires explicit permission.

Key example: The Pile removed Books3 (23 GB) after copyright challenges.

PII Removal

Personal information in training data can be memorised verbatim.

Techniques: regex redaction (email, phone, SSN), NER-based entity removal, differential privacy noise.

GDPR Art.17 ('right to erasure') creates tension with immutable model weights.

Copyright & NYT v. OpenAI

The New York Times sued OpenAI (Dec 2023) alleging verbatim reproduction of articles.

Central question: does training on copyrighted text constitute 'fair use'?

Outcome pending — reshaping how labs select and filter web data.

C O N C E P T

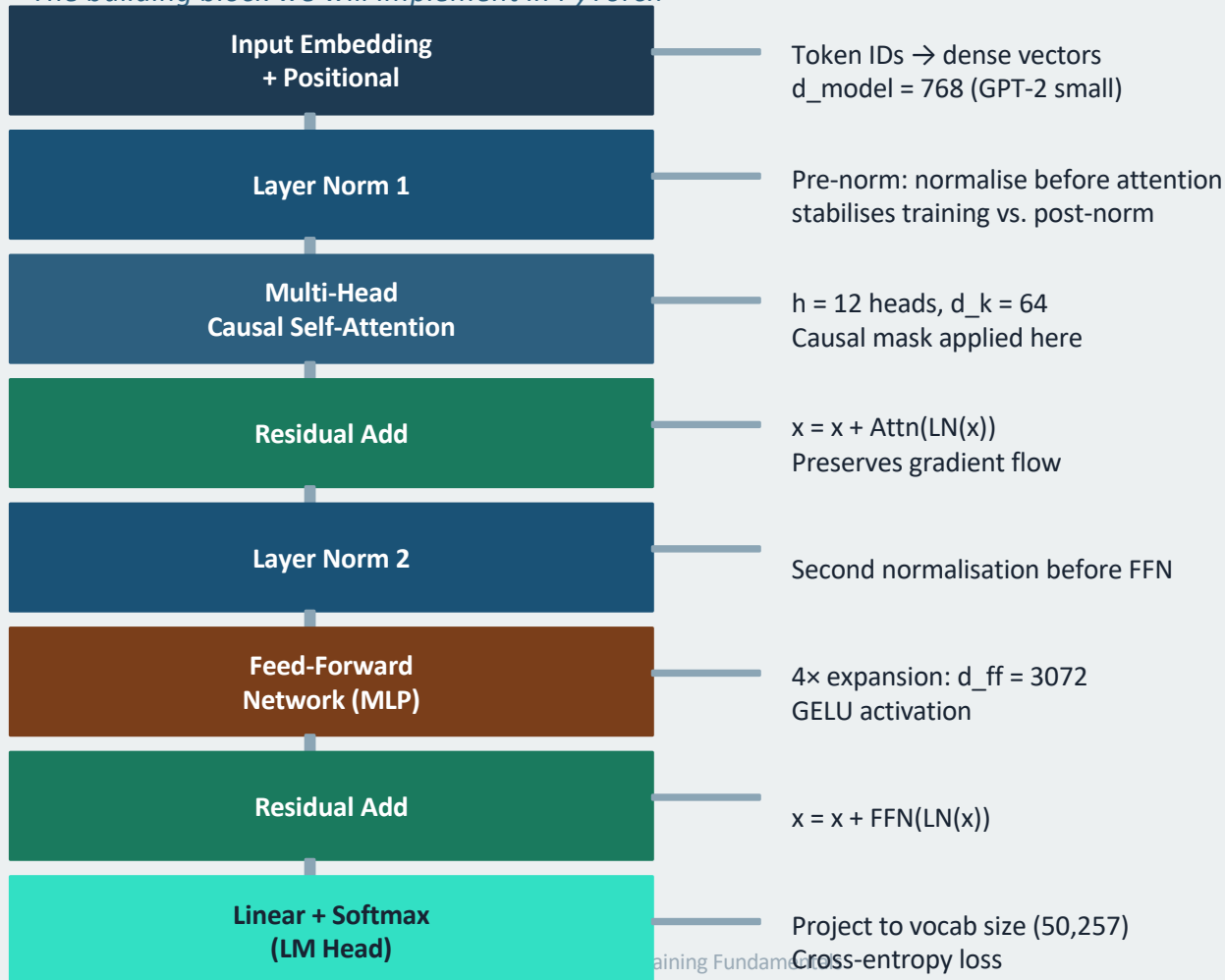
Implementing a Language Model from Scratch

Coding Lab — PyTorch · ~120M parameters · TinyShakespeare

6

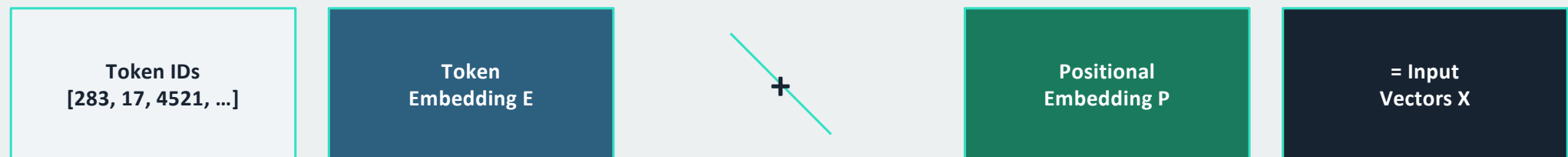
Transformer Block Anatomy

The building block we will implement in PyTorch



Token & Positional Embedding Layers

From discrete token IDs to continuous vector representations



Token + Positional Embedding in PyTorch

```
class EmbeddingLayer(nn.Module):
    def __init__(self, vocab_size, d_model, max_seq_len):
        super().__init__()
        self.tok_emb = nn.Embedding(vocab_size, d_model)
        self.pos_emb = nn.Embedding(max_seq_len, d_model)
    def forward(self, x):          # x: [B, T]
        pos = torch.arange(T, device=x.device).unsqueeze(0)  # [1, T]
        return self.tok_emb(x) + self.pos_emb(pos)            # [B, T, d_model]
```

Implementing Multi-Head Causal Self-Attention

The core computation of every transformer decoder

```
class CausalSelfAttention(nn.Module):
    def __init__(self, d_model, n_heads, dropout=0.1):
        super().__init__()
        assert d_model % n_heads == 0
        self.n_heads, self.d_k = n_heads, d_model // n_heads
        self.qkv_proj = nn.Linear(d_model, 3 * d_model, bias=False)
        self.out_proj = nn.Linear(d_model, d_model, bias=False)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        # x: [B, T, d_model]
        B, T, D = x.shape
        Q, K, V = self.qkv_proj(x).split(D, dim=-1)  # each [B, T, D]
        # Reshape for multi-head attention
        Q = Q.view(B, T, self.n_heads, self.d_k).transpose(1, 2)
        K = K.view(B, T, self.n_heads, self.d_k).transpose(1, 2)
        V = V.view(B, T, self.n_heads, self.d_k).transpose(1, 2)
        # Scaled dot-product + causal mask
        scores = (Q @ K.transpose(-2, -1)) / math.sqrt(self.d_k)  # [B,H,T,T]
```

The Causal Mask in Code

How we enforce autoregressive structure during training

```
# Apply causal mask to attention scores

mask = torch.tril(torch.ones(T, T, device=x.device))

mask = mask.bool() # lower-triangular

scores = scores.masked_fill(~mask, float('-inf'))

attn = F.softmax(scores, dim=-1) # -inf → 0

attn = self.dropout(attn)

out = attn @ V # [B, H, T, d_k]

out = out.transpose(1, 2).contiguous()

out = out.view(B, T, D) # merge heads

return self.out_proj(out) # [B, T, D]
```

torch.tril output (T=5)

1	0	0	0	0
1	1	0	0	0
1	1	1	0	0
1	1	1	1	0
1	1	1	1	1

Where mask = 0 → fill score with $-\infty$
 $\text{Softmax}(-\infty) = 0$ exactly
 → future tokens contribute nothing to output

Feed-Forward Network, LayerNorm & Residual Connections

The remaining components of a transformer block

```
class TransformerBlock(nn.Module):
    def __init__(self, d_model, n_heads, d_ff, dropout=0.1):
        super().__init__()
        self.attn = CausalSelfAttention(d_model, n_heads, dropout)
        self.ffn = nn.Sequential(
            nn.Linear(d_model, d_ff), # expand: 768 → 3072
            nn.GELU(),
            nn.Linear(d_ff, d_model), # contract: 3072 → 768
            nn.Dropout(dropout)
        )
        self.ln1 = nn.LayerNorm(d_model) # Pre-norm architecture
        self.ln2 = nn.LayerNorm(d_model)
    def forward(self, x):
        x = x + self.attn(self.ln1(x)) # Residual 1: attention
        x = x + self.ffn(self.ln2(x)) # Residual 2: feed-forward
        return x
```


Full Forward Pass — Assembling the Causal LM

Putting embedding + N transformer blocks + LM head together

```
class CausalLM(nn.Module):
    def __init__(self, vocab_size, d_model, n_layers, n_heads, d_ff, max_seq=1024):
        super().__init__()
        self.emb = EmbeddingLayer(vocab_size, d_model, max_seq)
        self.blocks = nn.ModuleList([TransformerBlock(d_model, n_heads, d_ff)
                                      for _ in range(n_layers)])
        self.ln_f = nn.LayerNorm(d_model) # final layer norm
        self.lm_head = nn.Linear(d_model, vocab_size, bias=False)
        # Weight tying: embedding and lm_head share parameters
        self.lm_head.weight = self.emb.tok_emb.weight

    def forward(self, x, targets=None): # x: [B, T]
        x = self.emb(x) # [B, T, d_model]
        for block in self.blocks:
            x = block(x)
        logits = self.lm_head(self.ln_f(x)) # [B, T, vocab_size]
        loss = F.cross_entropy(logits.view(-1, vocab_size), targets.view(-1))
        return logits, loss
```

Training Loop — Gradient Accumulation

How to simulate large batches on a single GPU

```

accum_steps = 8 # simulate batch_size × 8
optimizer.zero_grad()

for step, (x, y) in enumerate(train_loader):
    x, y = x.to(device), y.to(device)
    logits, loss = model(x, y)
    loss = loss / accum_steps # scale loss
    loss.backward()           # accumulate grads

    if (step + 1) % accum_steps == 0:
        # Clip gradients → stable training
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        scheduler.step()
        optimizer.zero_grad()
        log_loss(loss.item() * accum_steps)

```

Prof. Sashikumaar Ganesan · IISc Bangalore

Gradient Accumulation

Forward pass 1

$\text{loss}_1 / 8 \rightarrow \text{backward}$

Forward pass 2

$\text{loss}_2 / 8 \rightarrow \text{backward}$

...

...

Forward pass 8

$\text{loss}_8 / 8 \rightarrow \text{backward}$

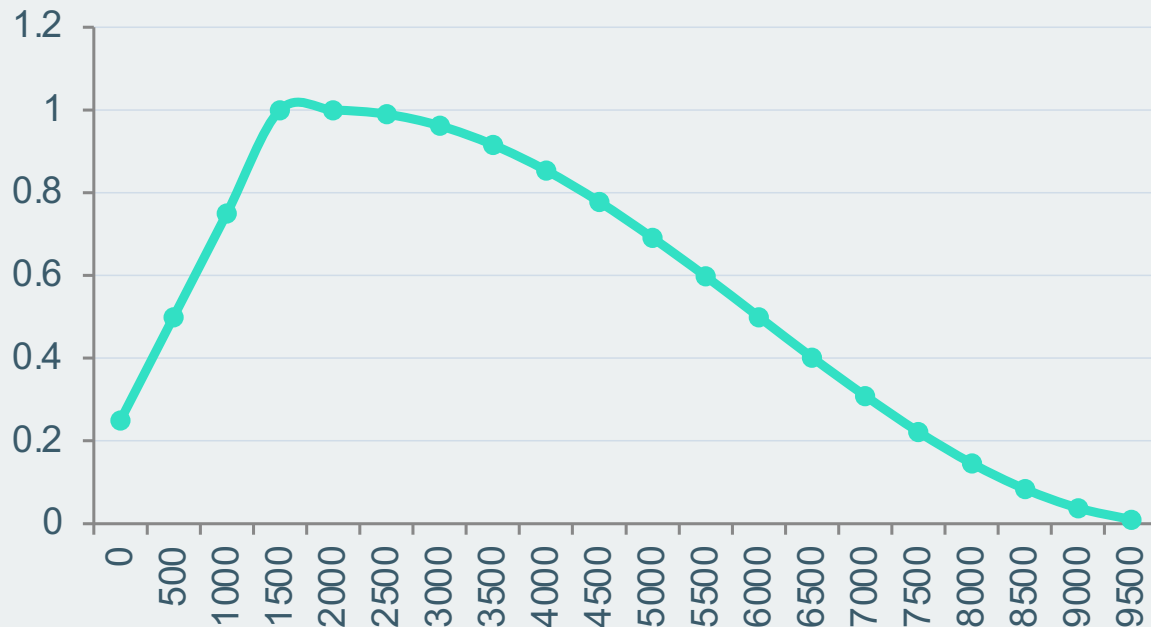
optimizer.step()

sum of 8 gradients applied

Effect: Equivalent to $\text{batch_size} \times \text{accum_steps}$ without extra VRAM

Learning Rate Warmup & Cosine Decay

Stable initialisation followed by principled learning rate annealing



← Warmup (linear rise) →

← Cosine decay →

Cosine LR Schedule

$$lr(t) = lr_max \times \frac{1}{2} [1 + \cos(\pi t/T)]$$

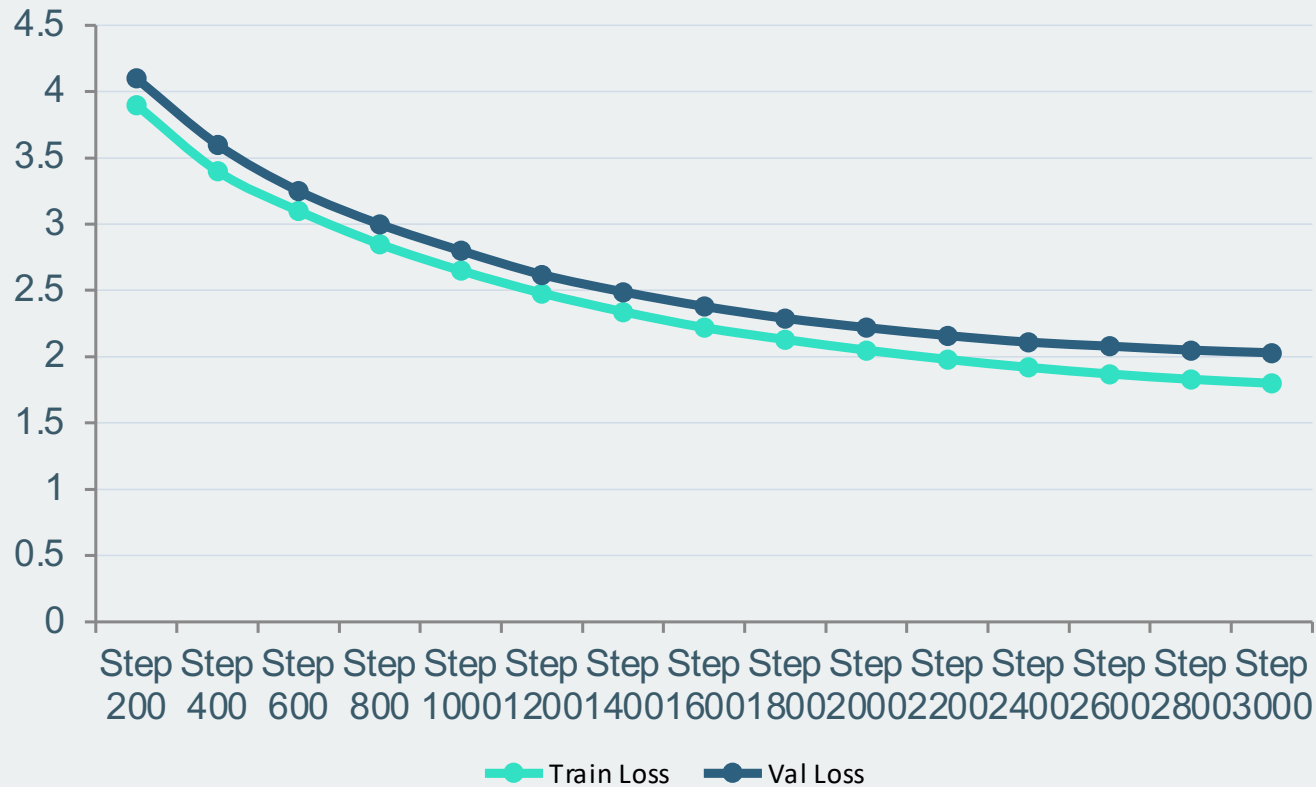
t = current step, T = total steps

```
def lr_lambda(step):
    if step < warmup_steps:
        return (step+1) / warmup_steps
    p = (step - warmup_steps) /
        (total_steps - warmup_steps)
    return 0.5 * (1 + math.cos(pi*p))

scheduler = LambdaLR(optimizer, lr_lambda)
```

Running on TinyShakespeare — Training & Validation Loss

What healthy CLM training curves look like



✓ Healthy sign

Train and val loss decreasing together — no overfitting

⚠ Watch for

Val loss flat while train drops → overfitting; reduce model size or add dropout

✓ Final PPL

Val loss $\approx 2.0 \rightarrow \text{PPL} \approx \exp(2.0) \approx 7.4$
(Random = PPL $\sim 50K$; GPT-2 = ~ 29 on test set)

Sub-Module 9.1 — Key Takeaways



1

Pre-training establishes foundation

Self-supervised next/masked token prediction on trillions of tokens creates transferable representations. Frontier models train for months on thousands of GPUs.

2

Objective → architecture → use case

CLM → decoder-only → generation. MLM → encoder-only → embeddings & classification. Prefix LM → encoder-decoder → seq2seq.

3

Causal mask enables parallel training

The lower-triangular mask enforces autoregression during training while keeping computation parallel. At inference, generation remains sequential.

4

Data quality matters as much as scale

Deduplication (MinHash), quality filtering (perplexity, classifier), and data mixing ratios all significantly impact capability. More data is not always better.

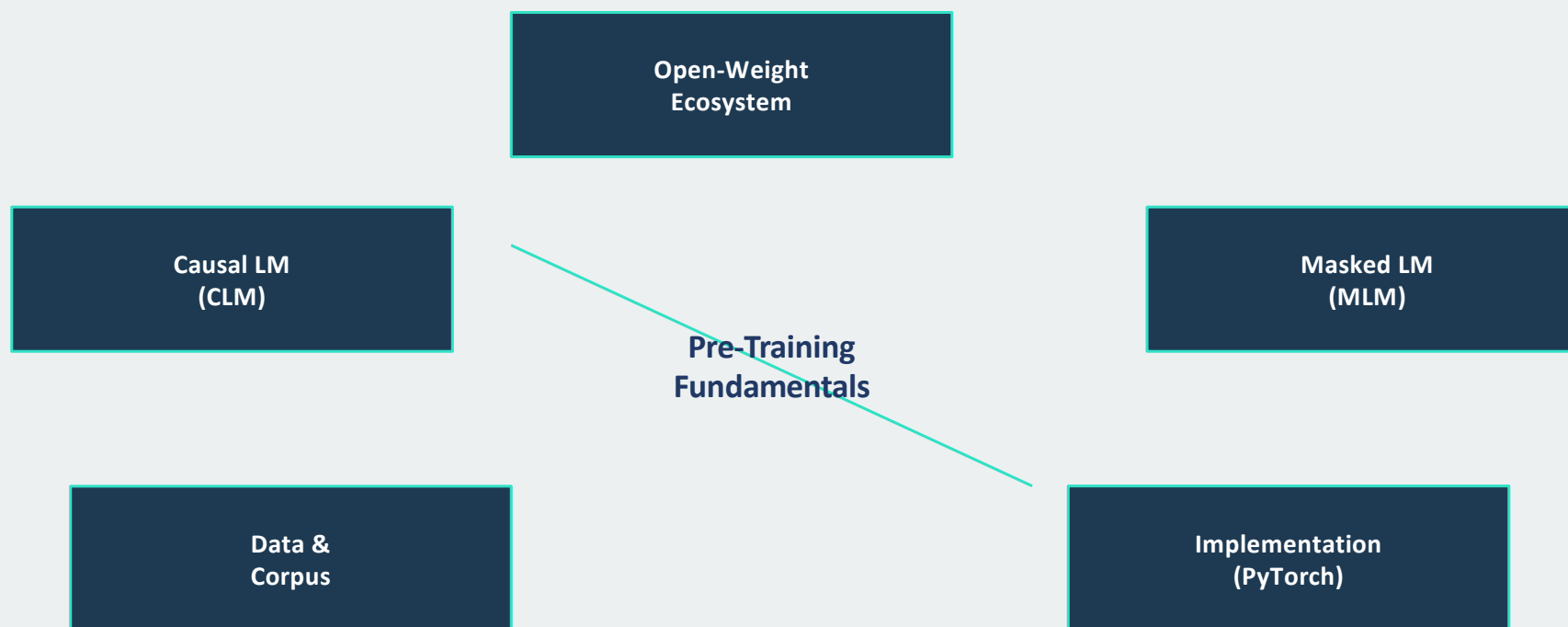
5

Implementation demystifies the black box

A ~120M parameter causal LM = embedding + N transformer blocks + LM head. Gradient accumulation, LR warmup, and weight tying are all standard practice.

Sub-Module 9.1 Concept Map

Everything connected — from objective to production



→ Transformer block · LR warmup · Gradient accumulation · Checkpoint saving →

Sub-Module 9.2

Infrastructure & Scale

GPU memory requirements · Quantization (INT4/INT8) · Mixed precision training · Gradient accumulation at scale · Flash Attention · Setting up distributed training

Hardware Landscape

A100 · H100 · RTX 4090
Memory walls

Quantization & Compression

FP16 → INT8 → INT4
Bitsandbytes · AWQ

Mixed Precision Training

FP16/BF16 forward
FP32 master weights

References



Key papers cited in Sub-Module 9.1

Authors	Title	Venue	ID
Devlin et al. (Google)	BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding	NAACL 2019	arXiv:1810.04805
Liu et al. (Meta)	RoBERTa: A Robustly Optimised BERT Pretraining Approach	arXiv 2019	arXiv:1907.11692
Radford et al. (OpenAI)	Language Models are Few-Shot Learners (GPT-3)	NeurIPS 2020	arXiv:2005.14165
Touvron et al. (Meta)	Llama 2: Open Foundation and Fine-Tuned Chat Models	arXiv 2023	arXiv:2307.09288
Gao et al.	The Pile: An 800GB Dataset of Diverse Text for Language Modeling	arXiv 2020	arXiv:2101.00027
Lee et al.	Deduplicating Training Data Makes Language Models Better	ACL 2022	arXiv:2107.06499